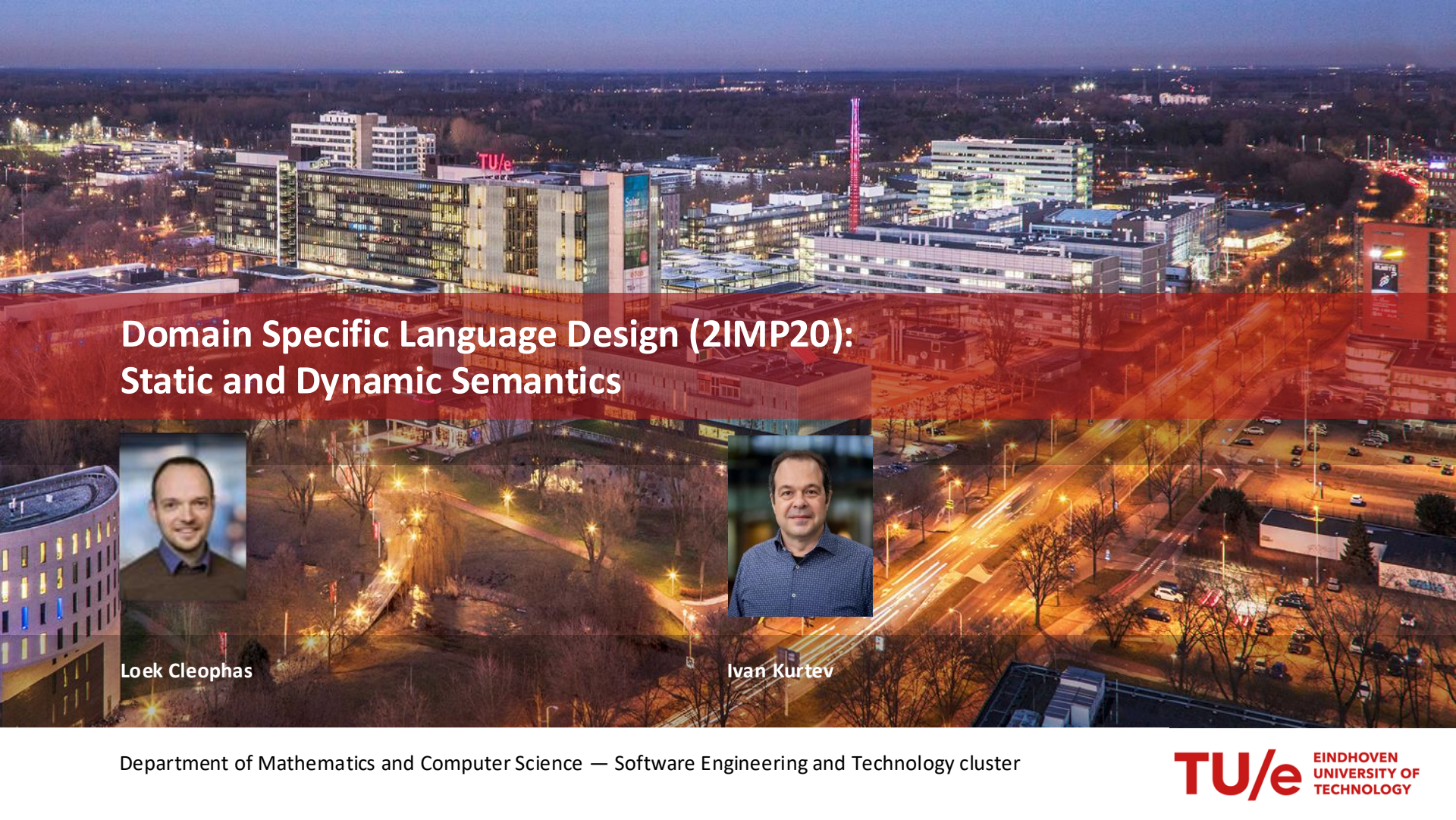




Domain Specific Language Design (2IMP20): Static and Dynamic Semantics



Loek Cleophas



Ivan Kurtev

Implementation of a DSL

Definition of a (programming) language involves:

- abstract syntax
- concrete syntax:
 - textual syntax
 - graphical syntax
- semantics:
 - static
 - dynamic

Implementation of a DSL

Definition of a (programming) language involves:

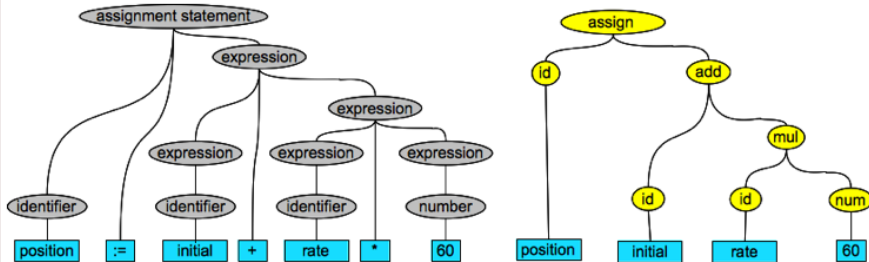
- abstract syntax
- concrete syntax:
 - textual syntax
 - graphical syntax
- semantics:
 - static
 - dynamic

Semantics

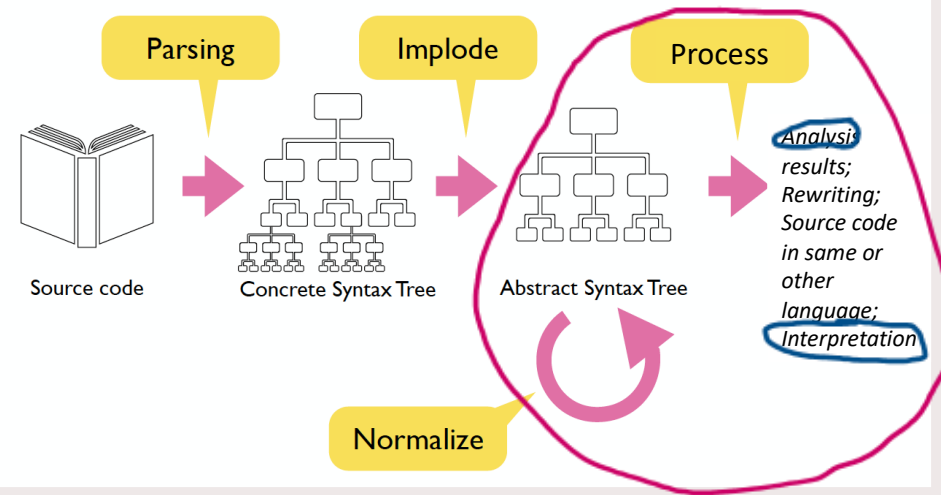
For the input sentence

position := initial + rate * 60

the parse tree (left) and abstract syntax tree (right) may look as follows:



(Source: <https://www.rascal-mpl.org/docs/Rascalopedia/ParseTree/>)



Dynamic semantics

Semantics *represents the intended meaning* of a language and language constructs

Semantics is a *means to represent our understanding* of a model/program (**what it does**) and

To *communicate our understanding* to other entities

To understand what happens in a computer/machine *when a program/model is executed*

Dynamic semantics

Not every language is suitable for describing dynamic semantics

- C and Java are not really suited for describing the dynamic semantics, because of lack of a formal description

Dynamic semantics *ideally* is described in a *formal language (formal semantics)* because:

- *ambiguities and inconsistencies can be detected* in a model which appears to be “ok”
- this is the *basis for analysis, validation and verification*, but also *implementation*

Dynamic semantics

Dynamic semantics in practice

- Semantics of languages is often defined by *translation* or *interpretation*
- Languages evolve over time and semantic ambiguities and conflicts may be introduced

Dynamic semantics

Formal dynamic semantics

- Modern systems models are
 - complex
 - (possibly) at a high level of abstraction
- Reasoning about the models using rigorous methods
 - need to find existing *ambiguities and inconsistencies*
 - need to keep a (modeling) language “*clean and simple*”
- Formal semantics allows for
 - analysis, validation and verification, but also (correct) implementation
 - model/program comparison, thus optimization, modification

Operational semantics

- specifies how (step-by-step) program/model executes
- specifies how states/configurations are modified during the execution
- all possible executions are generated
- underlying model is the model of *transition systems* (a program/model execution is turned into a transition system)

Operational semantics

Big-step operational semantics deals with *relations* over program fragments and data structures to *represent values and stores*:

- *Values* represent *values of evaluation*
- *Stores* are *maps from variable names to values*

Judgements are claims of relationships, a judgement is a basic formula (a claim to be proven based on the operational semantics specification):

- $\text{evaluate}(e, v)$: a judgement of the evaluation of **expression** e to value v ;
- $\text{execute}(s, m, m')$: a judgement of the execution of an **imperative statement** s for a store m and a resulting store m' ;
- $\text{evaluate}(fs, m, e, v)$: a judgement of the evaluation of expression e in a **functional program** to value v relative to a given collection of functions fs and an environment m .

Operational semantics

An operational semantic specification consists of inference rules:

$$\frac{P_1 \quad \dots \quad P_n}{C} \quad [l]$$

where $P_1, \dots, P_n$ and C are judgements (*premises + conclusion*) and l is simply a label used as reference.

If $n = 0$, the judgement is an axiom

$$C \quad [l]$$

Operational semantics

The definition of operational semantics is **syntax-driven**:

- the conclusion applies to a specific syntactic construct
- the premises may apply to subterms of this construct or terms formed over it

Example:

`evaluate (zero, zero)                      [zero]`

`evaluate (e, n)`
`=====                      [succ]`
`evaluate (succ (e) , succ (n) )`

Operational semantics

Operational semantics of BTL (from book of Lämmel):

`evaluate (true, true)` `[true]`

`evaluate (false, false)` `[false]`

`evaluate (zero, zero)` `[zero]`

`evaluate (e, n)`
===== `[succ]`
`evaluate (succ (e), succ (n))`

`evaluate (e, zero)`
===== `[pred1]`
`evaluate (pred (e), zero)`

`evaluate (e, succ (n))`
===== `[pred2]`
`evaluate (pred (e), n)`

Operational semantics

```
    evaluate(e, zero)
===== [iszero1]
evaluate(iszero(e), true)
```

```
    evaluate(e, succ(n))
===== [iszero2]
evaluate(iszero(e), false)
```

```
    evaluate(e0, true)      evaluate(e1, v1)
===== [if1]
evaluate(if(e0, e1, e2), v1)
```

```
    evaluate(e0, false)     evaluate(e2, v2)
===== [if2]
evaluate(if(e0, e1, e2), v2)
```

Operational semantics – complete set of rules

Operational semantics of BTL (from book of Lämmel):

```
evaluate(true, true)      [true]

evaluate(false, false)    [false]

evaluate(zero, zero)      [zero]

      evaluate(e, n)
===== [succ]
evaluate(succ(e), succ(n))

      evaluate(e, zero)
===== [pred1]
evaluate(pred(e), zero)

      evaluate(e, succ(n))
===== [pred2]
evaluate(pred(e), n)
```

```
      evaluate(e, zero)
===== [iszero1]
evaluate(iszero(e), true)

      evaluate(e, succ(n))
===== [iszero2]
evaluate(iszero(e), false)

evaluate(e0, true)      evaluate(e1, v1)
===== [if1]
evaluate(if(e0, e1, e2), v1)

evaluate(e0, false)     evaluate(e2, v2)
===== [if2]
evaluate(if(e0, e1, e2), v2)
```


Operational semantics

Derivation tree for evaluating a BTL expression:

```
evaluate (zero, zero)          [zero]

===== [iszero1]

evaluate (iszero(zero), true)    evaluate (succ(zero), succ(zero))

===== [if1]

evaluate (if (iszero(zero), succ(zero), zero), succ(zero))

===== [pred2]

evaluate (pred (if (iszero(zero), succ(zero), zero)), zero)
```

Operational semantics

Derivation tree for evaluating a BTL expression:

```
evaluate (zero, zero)          [zero]

===== [iszero1]

evaluate (iszero (zero), true)          evaluate (succ (zero), succ (zero))

===== [if1]

evaluate (if (iszero (zero), succ (zero), zero), succ (zero))

===== [pred2]

evaluate (pred (if (iszero (zero), succ (zero), zero)), zero)
```

Operational semantics

Derivation tree for evaluating a BTL expression:

```

    evaluate (zero, zero)          [zero]          evaluate (zero, zero)          [zero]
===== [iszero1] ===== [succ]
    evaluate (iszero (zero), true)          evaluate (succ (zero), succ (zero))
===== [if1]
    evaluate (if (iszero (zero), succ (zero), zero), succ (zero))
===== [pred2]
    evaluate (pred (if (iszero (zero), succ (zero), zero)), zero)
```

Operational semantics

Proof of a big-step judgement has a tree-like shape, so called *derivation trees*:

- Each node in a derivation tree is an instance of a conclusion of an inference rule
 - Subtrees of a node are instances of the premises of the same rule
- Leaf nodes of the tree are instance of axioms
- Instances are metavariables in the rules consistently replaced by phrases of data structures of the appropriate sort

Questions?



A PICO interpreter based on big-step “operational” semantics

Definition of an interpreter for PICO:

- Natural variables are initialized to 0
- String variables are initialized to “”
- Variable on lhs of assignment gets value of Rhs
- Variable (in expression) evaluates to its current value
- Test in while and if-then equal to 0 => false
- Test in while and if-then not equal to 0 => true

The Pico interpreter “transforms” a Pico program to the output it generates, by stepwise reduction in a syntax directed manner

This interpreter implements the *big-step “operational” semantics*

A PICO interpreter — Values

```
data PicoValue =  
  natval(int n) |  
  strval(str s) |  
  errorval(loc l, str msg); ●
```

errorval denotes error messages
and their locations

A PICO interpreter — Value environment a.k.a. store

Alias VENV

```
map[[PicoId, PicoValue]]
```

A PICO interpreter — Evaluating expressions

```
PicoValue evalExp(exp:natCon(int N), VENV env) = natval(N);

PicoValue evalExp(exp:strCon(str S), VENV env) = strval(S);

PicoValue evalExp(exp:id(PicoId Id), VENV env) =
  env[Id]? ? env[Id] : errorval(exp@location, "Uninitialized variable <Id>");

PicoValue evalExp(exp:add(EXP E1, EXP E2), VENV env) =
  (natval(n1) := evalExp(E1, env) &&
   natval(n2) := evalExp(E2, env)) ? natval(n1 + n2)
    : errorval(exp@location, "+ requires natural arguments");
```

A PICO interpreter — Evaluating expressions

```
PicoValue evalExp(exp:sub(EXP E1, EXP E2), VENV env) =  
  (natval(n1) := evalExp(E1, env) &&  
   natval(n2) := evalExp(E2, env)) ? natval(n1 - n2)  
                                     : errorval(exp@location,  
                                                "- requires natural arguments");  
  
PicoValue evalExp(exp:conc(EXP E1, EXP E2), VENV env) =  
  (strval(s1) := evalExp(E1, env) &&  
   strval(s2) := evalExp(E2, env)) ? strval(s1 + s2)  
                                     : errorval(exp@location,  
                                                "|| requires string arguments");
```

A PICO interpreter — Evaluating lists of statements

```
VENV evalStats(list[STATEMENT] Stats1, VENV env) {  
  for (S <- Stats1){  
    env = evalStat(S,env);  
  }  
  return env;  
}
```

A PICO interpreter — Evaluating a statement

```
VENV evalStat(stat:asgStat(PicoId Id, EXP Exp), VENV env) {
    env[Id] = evalExp(Exp, env);
    return env;
}

VENV evalStat(stat:ifElseStat(EXP Exp, list[STATEMENT] Stats1,
                             list[STATEMENT] Stats2),
              VENV env) =
    evalStats(evalExp(Exp, env) != natval(0) ? Stats1 : Stats2, env);

VENV evalStat(stat:whileStat(EXP Exp, list[STATEMENT] Stats1),
              VENV env) {
    while(evalExp(Exp, env) != natval(0)){
        env = evalStats(Stats1, env);
    }
    return env;
}
```

Evaluating declarations and a Pico program

```
VENV evalDecls(list[DECL] Decls) =  
  ( Id : (tp == demo::lang::Pico::Abstract::natural() ?  
          natval(0) : strval(""))  
    | decl(PicoId Id, TYPE tp) <- Decls);  
  
public VENV evalProgram(PROGRAM P){  
  if(program(list[DECL] Decls, list[STATEMENT] Series) := P){  
    VENV env = evalDecls(Decls);  
    return evalStats(Series, env);  
  } else  
    throw "Cannot happen";  
}
```

Translational Semantics

An alternative route to define the semantics of a DSL is to generate code for some general-purpose programming language:

- Java
- C
- Haskell

No real guarantee with respect to completeness and consistency because of lack of precise semantics of many general-purpose programming languages

Dynamic semantics

Research related topics

- Integration of (static and dynamic) semantics in a language workbench
- Size of semantic building block:
 - on the level of “assembler instructions”
 - on the level of “semantic patterns”: queues, state machines, channels, etc.
- Domain specific languages to describe both static and dynamic semantics

Questions?



Background information

Reading:

- Chapter 3, 4 and 5 of *Introduction to Compiler Design*
Covers scoping, interpretation, type checking
- Chapter 8 and 9 of *Software Languages*
Covers operational semantics + type checking

Lookahead

- 2026/06/03: Ivan (code generation)
- 2026/06/05: DSL design (the bigger picture)
- 2026/06/10: guest lecture Eugen Schindler, Canon Production printing
- 2026/06/12: guest lecture, tbd
- 2026/06/17 and 2025/06/19: backup slots